Dokumentation

Hausautomatisierung



Enes Türksever

Inhaltsverzeichnis

1. Einleitung.....3

2. Bauteile.....4

2.1 Raspberry Pi.....4

2.2 Extensionsboard.....4

2.3 Breadboard.....4

2.4 Jumper Kabel.....5

2.5 Widerstand.....5

2.6 Dioden LED.....5

2.7 Knopf.....5

2.8 Bewegungssensor.....5

2.9 Touchsensor.....6

2.10 Lichtsensor.....6

2.11 Stepmotor.....7

3 Umsetzung.....7

3.1 Planung.....7

3.2 Umsetzungsphase der Touchbar gesteuerten Haus Belichtung.....8

3.2.1 Theoretische Umsetzung.....8

3.2.2 Code.....8

3.2.3 Probleme.....9

3.3Umsetzungsphase des Automatischen Außen Strahlers.....10

3.3.1 Theoretische Umsetzung.....10

3.3.2 Code.....10

3.3.3 Probleme.....11

3.4 Umsetzungsphase der Automatisierte und manuell steuerbare Jalousien.....11

3.4.1 Theoretische Umsetzung.....11

3.4.2 Code.....12

3.4.3 Probleme.....14

4. Fazit.....14

5. Literaturverzeichnis.....15

1. Einleitung

Unsere Familie hat sich kürzlich ein Haus gekauft und mein Vater ist begeistert von der zunehmenden Popularität von Smarthome-Technologien. Er hat mir den Auftrag gegeben, ein kleines Smarthome-System mit einem Raspberry Pi nachzubauen, um zu sehen, wie es funktioniert und welche Vorteile es bietet. Mit dieser Aufgabe wurde mir die Möglichkeit gegeben, meine Fähigkeiten im Bereich der Elektrotechnik und Programmierung zu verbessern und gleichzeitig das Haus unserer Familie in ein intelligentes Zuhause zu verwandeln. Die Planung und Umsetzung des Smarthome-Systems war ein wichtiger Schritt in diesem Projekt. Eine gründliche Planung half dabei, alle notwendigen Komponenten zu identifizieren und ein solides Fundament für den Bau des Systems zu schaffen. In der Planungsphase wurde auch die Funktionalität des Systems diskutiert und entschieden, welche Bauteile benötigt werden. Eine der wichtigsten Funktionen des Smarthome-Systems war die Steuerung von Lichtern durch Bewegungssensoren. Ein weiteres wichtiges Element des Smarthome-Systems waren automatisierte Jalousien. Der Touchsensor für die Beleuchtung im Haus war eine neuer, aber wichtige Funktion des Smarthome-Systems. Während der Umsetzung des Smarthome traten jedoch auch einige Probleme auf. Einige der Bauteile erwiesen sich als schwierig zu installieren oder zu konfigurieren, was zu Verzögerungen im Projekt führte. Auch die Programmierung der verschiedenen Bauteile war eine Herausforderung, da sie alle miteinander kommunizieren mussten, um das Smarthome-System ordnungsgemäß zu betreiben. Trotz dieser Herausforderungen war die Fertigstellung des Smarthome-Systems ein wichtiger Meilenstein in unserem Projekt. Das System bot zahlreiche Vorteile, darunter eine verbesserte Energieeffizienz, erhöhte Sicherheit und eine verbesserte Benutzerfreundlichkeit. Darüber hinaus eröffnete das Projekt neue Möglichkeiten, um unsere Fähigkeiten in der Elektronik und Programmierung zu verbessern. Ein weiterer wichtiger Aspekt des Projekts war die Zusammenarbeit innerhalb der Familie. Jeder trug seine Fähigkeiten und Ideen bei, um das Smarthome-System zu verbessern und es zu einem Erfolg zu machen. Mein Vater half bei der Auswahl der Bauteile und der Planung des Systems, während ich mich auf die Programmierung und die technischen Details konzentrierte.

In der folgenden Dokumentation werden wir uns ausführlicher mit den einzelnen Bauteilen und Funktionen des Smarthome-Systems beschäftigen. Wir werden auch die verschiedenen

Herausforderungen und Probleme diskutieren, auf die ich während des Projekts gestoßen bin, und wie ich sie gelöst haben. Schlussendlich wird es noch ein allgemeines Fazit geben.

2. Bauteile

2.1 Raspberry Pi

Was ist ein Raspberry Pi eigentlich?
Grob gesagt kann man sagen das der Raspberry Pi nichts anderes ist als ein vollwertiger Computer. Hierbei solltest du dir aber kein klassischen Computer vorstellen (siehe Abb.1), wie du ihn sicherlich zuhause auf dem Schreibtisch hast, dieser hat einige Funktion, die man vor allem in einem Projekt wie in diesem Falle wirklich gebrauchen kann. Der Raspberry Pi überzeugt nicht nur mit seinem eher geringeren Preis, sondern auch mit seiner Leistung. Die Prozessoren sind schnell, genügend RAM für gleichzeitige Arbeit und darüber hinaus besitzt er über eine Vielzahl von Ein und Ausgängen. Eine Besonderheit sind die GPIO-Pins, die vor allem in unserem Projekt essenziell von Nöten waren, um sowas wie ein Smarthome überhaupt nachzustellen. Zurück auf unser Projekt betrachtet kann man den Raspberry Pi als Steuerzentrale sehen, da er die Sensoren und Aktoren überwacht und steuert.

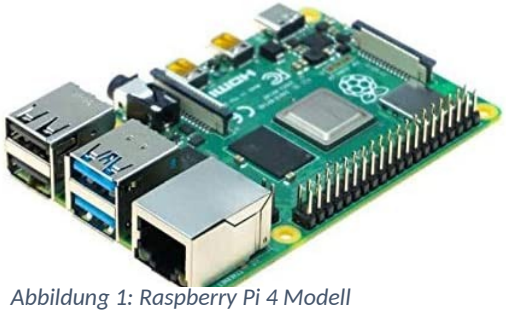


Abbildung 1: Raspberry Pi 4 Modell

2.2 Extensionsboard

Ein Extensionboard stellt eine Erweiterung der bereits vorhandenen Pins des Raspberry Pi dar. Neben den standardmäßigen GPIO-Pins bietet es weitere Anschlussmöglichkeiten, wie beispielsweise 3,3V, 5V und Ground Pins. Durch die eindeutige Beschriftung der Pins auf dem Extensionboard wird die Arbeit erleichtert und eine bessere Übersicht gewährleistet. Insgesamt vereinfacht das Extensionboard die Arbeit und steigert die Effizienz bei der Erweiterung des Raspberry Pi.

2.3 Breadboard

Das Breadboard, auf Deutsch auch Steckbrett genannt ist eine Kunststoffplatte mit einer Vielzahl von Löchern. Die Löcher sind dafür da damit die ganzen Aktoren und Sensoren nicht verlötet werden müssen, sondern einfach über ein Jumper Kabel verbindet werden können. Der Vorteil hierbei besteht auch, dass man die Verbindung schnell, einfach prüfen und testen kann und je nachdem umstecken kann falls Fehler auftreten. Meine Empfehlung wäre in solchen Projekten wie in meinem Falle mit vielen verschiedenen Modulen, pro Modul ein Breadboard zu nutzten. Das dient der allgemeinen Übersicht.

2.4 Jumper Kabel

Jumper Kabel sind elektrische Kabel mit Steckverbindung an beiden Enden, diese werden in der Elektrotechnik genutzt um Elektronische Komponenten miteinander zu Verbinden. Natürlich gibt es von diesen Kabeln noch viele Varianten was Länge, Farbe und der Anschlüsse angeht. Im ersten Moment Würde man jedoch sie mit den GPIO-Kabel verwechseln, jedoch liegt der unterschied darin das die Jumper Kabel mit theoretisch jeder Elektronik in Verbindung gesetzt werden kann, wobei die die GPIO-Kabel nur mit ihren eigenen Input/Output Pins fungieren.

2.5 Widerstand

Der Elektronische Widerstand ist in der Elektrotechnik ein wichtiger Bestandteil und ist in fast jedem mit Elektrizität gesteuertem Gerät verbaut. Es handelt sich dabei um Regulator des Stromkreises. Er reguliert oder begrenzt dabei, indem er den Stromfluss erhöht. Dieser wird in der Einheit Ohm (Ω) gemessen. Bei der nachgebildeten Hausautomatisierung hat er bei der beim Bewegungsmelder sein gebrauch gefunden, da hier eine LED verbaut ist.

2.6 Dioden LED

Eine Dioden LED ist eine Halbleiter Lichtquelle, die licht emittiert, wenn Strom durch sie fließt im Gegensatz zu herkömmlichen Glühlampen wandeln LED elektrische Energie direkt in licht um anstatt wärme zu erzeugen. Im Projekt wurde sie als Beleuchtung benutzt

2.7 Knopf

Der 4-Pin-Knopf ist ein einfacher Knopf, der normalerweise als Drucktaster verwendet wird. Er hat vier Anschlüsse: zwei für die Stromversorgung und zwei für die Schaltung. Wenn der Knopf gedrückt wird, werden die beiden Stromversorgungsanschlüsse verbunden, was es dem Strom ermöglicht, durch die Schaltung zu fließen. Sobald der Knopf losgelassen wird, werden die Stromversorgungsanschlüsse wieder getrennt und die Schaltung unterbricht den Stromfluss.

2.8 Bewegungssensor

In meinem Projekt wurde der HC-SR501 PIR Bewegungssensor benutzt, dieser funktioniert über Infrarot (siehe ABB.2) und wird mit 5V betrieben. Es ist aber kein aktiver, sondern passiver Sensor, dieser erkennt durch die Wärme Strahlung von Objekt oder Mensch eine Veränderung und gibt somit ein Input an in unserem Falle dem

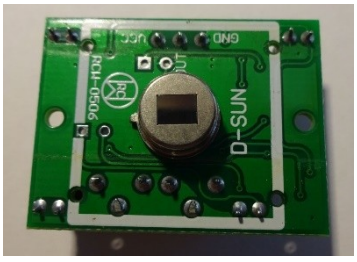


Abbildung 2: Das innere des Bewegungssensor (Infrarotsensor)

Raspberry Pin zurück. Genauer gesagt lauft es über den IC-Pin, wo das Signal bei Erkennung von Bewegung verarbeitet wird. Wobei das Ausgangssignal ein standardmäßigen 3.3 V wiedergibt.

Dieses läuft je nach Einstellung des Sensors 3-15 Sekunden, diese kann man über den linken (siehe ABB. 3) der beiden vorhandenen Regler einstellen. Eine weitere Funktion und der rechte Regler sind dafür da, um die Empfindlichkeit des Sensors einzustellen. Der Empfindlichkeits Regler bestimmt, wie stark eine Änderung der Wärmestrahlung wargenommen werden soll um letztendlich als Bewegung anerkannt zu werden.



Abbildung 3: der komplette Bewegungssensor

2.9 Touchsensor

Heutzutage ist sowas wie ein Touchsensor ein Muss in Smarthome, aus diesem Grund habe ich den Touch-Sensor TTP223b in mein Smarthome verbaut. Dieser misst von einer bestimmt verbauten Elektrode und der Oberfläche, wenn eine Berührung auf der metallischen oder auch leitfähigen Oberfläche erkannt wird. Bei Berührung des Sensors wird ein Teil der im Körper vorhanden elektrischen Ladung auf die Elektrode übertragen. Durch diesen Prozess wird zwischen der Kapazität und der Elektrode die Leitfähigkeit der Oberfläche verändert was im Endeffekt dazu führt das der Sensor reagiert. In meinem Projekt wurde dies als Lichtschalter verwendet.

2.10 Lichtsensor

Der LM393 Lichtsensor ist ein sowohl analoger, aber auch digitaler Sensor, der je nach Helligkeit der Umgebung elektrische Signale abgibt. Der Sensor besitzt eine Photodiode, diese reagiert nicht nur auf Licht, sondern auch Operationsverstärker, dieser verstärkt es und vergleicht es dabei noch mit einer Referenzspannung. Um die genauer zu verstehen, wie die Photodiode funktioniert muss man verstehen das er ein elektrisches Signal erzeugt was proportional zur Umgebungshelligkeit ist und wird darauf hin verglichen. Wenn hierbei das Signal die Referenzspannung übersteigt, wird am Ausgang ein High-Signal ausgegeben und ein Low-Signal beim Gegenteil. Der Sensor kann nur mit einer 3,3 bis 5V betrieben werden.

2.11 Stepmotor

Die 28BYJ-48 Schrittmotoren sind kleine Gleichstrommotoren, die mit 5V betrieben werden, diese sind präzise zu bewegen und Positionierbar. Wie im Namen enthalten, arbeiten diese nicht mit dauerhaften Drehungen, sondern sogenannten “Stepps“ also Schritten. Um den Roter des Schrittmotors in eine bestimmte Position zu bringen, verfügt dieser über vier Spulen, die nacheinander angesteuert werden. Davon wird jede Spule wechselweise in Form eines Stromfluss in eine Richtung magnetisiert (siehe ABB.4), welche entweder positiv oder negativ ist, dies führt zu einem Drehmoment auf dem Rotor. Dieser dreht sich zuletzt, um in die nächste Position zu gelangen und selbstverständlich die Ausrichtung der Spulen zu folgen. Durch Frequenzen und Abfolge der Signale, wird die Geschwindigkeit, wie auch die Richtung des Motors gesteuert, die an die Spulen gesendet werden. Er besitzt hierzu insgesamt fünf Drähte, von denen vier für die Spulen zuständig und einen für die gemeinsame Verbindung sind.

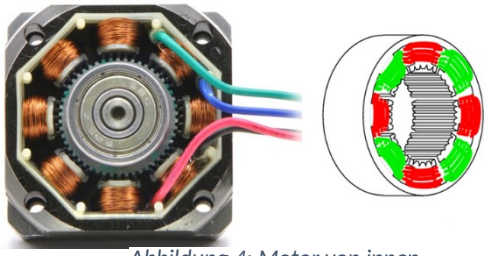


Abbildung 4: Motor von innen

3 Umsetzung

3.1 Planung

Wie in der Einleitung schon kurz erwähnt wurden für dieses Projekt eine sorgfältige Planung erstellt. In dieser Planung wurden sich drei Fragen gestellt. Erstens: Was wollen wir umsetzten? Zweitens: Was wird benötigt? Zuletzt Drittens: Wie wollen wir das Endprodukt am besten darstellen? Die erste Frage wurde sich erstmals gemeinsam mit der ganzen Familie gestellt, dazu schrieb jeder 5 Punkte auf, die in ein Smarthome müssen. Hierbei erkannt wir schnell das fast drei Funktionen bei jedem drauf standen. Erstens und der wichtigste Punkt wären automatisierte Jalousien, diese Funktion wollte keiner missen. Zweitens war ein Außen Strahler, der schon ein Standard vor jeder Haustüre ist. Schlussendlich wollten wir noch etwas Neueres ausprobieren und gewöhnliche Lichtschalter mit einer Touchbar tauschen, um die Lichter anzusteuern. Nach dieser Frage wurde von meiner Seite aus viel recherchiert um zuschauen welche Produkte wir für diese Konstruktion brauchen. Hierbei habe ich mich für simple Produkte entschieden, die aber zuverlässig sind. Diese wurden vorab schon genannt und kurz beschrieben und werden in den einzelnen Funktion noch etwas genauer erklärt und

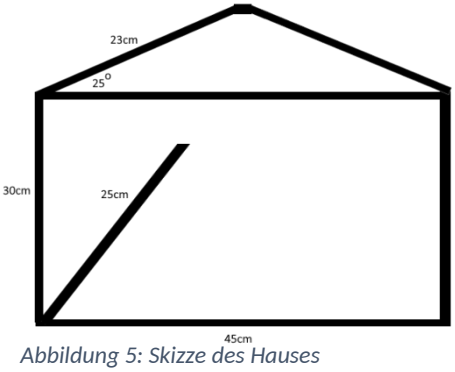


Abbildung 5: Skizze des Hauses

wofür sie zuständig sind. Letztendlich kamen wir wieder mit meinem Vater zusammen, um zu besprechen, wie wir dieses Smarthome darstellen wollen am Ende entschieden wir uns für Holz. Wir schauten uns daraufhin Puppenhäuser und Vogelhäuser an, dadurch geschuldet, dass sie von den Proportionen her in unser Projekt passen. Nach der Recherche fingen wir dann an am Computer eine kleine Skizze zu zeichnen und die Masse einzutragen (siehe ABB.5).

3.2 Umsetzungsphase der Touchbar gesteuerten Haus Belichtung

3.2.1 Theoretische Umsetzung

Für die Umsetzung dieser Funktion brauchen wir folgende Materialien: 1xBreadboard, 12 Jumper Kabel, 4x Dioden LED und ein Lichtsensor. Erst fangen wir damit an den Touch-Sensor TTP223b zu verkabeln hierzu stecken wir den GND-Pin vom Sensor an unser Extensionboard an, hierzu können wir einer der zahlreichen Ground Pins benutzten. Als nächsten wird dann der VCC-Pin anschlossen, da dieser mindestens mit 3,3V betrieben werden muss stecke ich diesen an einer der 5V Pins an. Zu guter Letzt wird der SIG-Pin noch mit dem Extensionsboard verbunden, hierfür muss man jedoch einer der GPIO-Pins nehmen, da dieser ein Input-Signal weitergeben muss, ich nutzte hierfür den GPIO-Pin 21. Nun bestimme ich noch den Output Signal der später bei Berührung des Touchsensor ein Signal an die Diodenleds weitergeben soll, dazu benutzte ich den nebenliegenden GPIO-Pin 20. Jetzt Schließen wir nun die 4x Diodenleds ans Breadboard mit einem Abstand von 5 “Löchern“, wobei man beachten muss das die kürzere Seite des Pins der LED mit einem Ground-Pin verbunden werden muss und die längere mit dem Output Signal auf Pin 20. Um viel Verkabelung zu sparen habe ich am Breadboard die – Seite mit Ground-Pins verbunden und die + Seite mit dem Output des GPIO-Pin 20. Schlussendlich wird man nun im Haus die kompletten LED mit diesem einem Touchpad steuern können.

3.2.2 Code

```
def light_thread():
    global Value1
    global Value
    while True:
        touch = GPIO.input(21)
        if touch == 1 and Value == 0:
            print("Licht an")
            Value = 1
            Value1 = Value1 + 1
            GPIO.output(led, GPIO.HIGH)
        elif touch == 1 and Value == 1:
            print("Licht aus")
            Value = 0
            Value1 = Value1 - 1
            GPIO.output(led, GPIO.LOW)
        time.sleep(0.3)
```

Abbildung 6: Code vom Touchbar gesteuerten LED-Beleuchtung

Um auf Variablen außerhalb der Funktion zugreifen zu können, verwendet die Funktion die Schlüsselwörter "global Value1" und "global Value". Dadurch wird sichergestellt, dass Änderungen an

diesen Variablen innerhalb der Funktion auch außerhalb der Funktion sichtbar sind. Innerhalb der Funktion wird eine Endlosschleife definiert, die ständig den Status des Touchsensor abruft. Dies geschieht mithilfe des Befehls "GPIO.input(21)", der den Wert des Pins 21 des Raspberry Pi ausliest. Wenn dieser Wert 1 ist und "Value" (eine der globalen Variablen) 0 ist, wird das Licht eingeschaltet. In diesem Fall wird "Value" auf 1 gesetzt, um anzuzeigen, dass das Licht eingeschaltet ist. Außerdem wird "Value1" um 1 erhöht, um die Anzahl der Einschaltvorgänge zu zählen. Schließlich wird die GPIO-Ausgabe des LED-Pins auf "HIGH" gesetzt, um das Licht einzuschalten. Wenn der Wert des Lichtsensors 1 ist und "Value" bereits 1 ist, wird das Licht ausgeschaltet. In diesem Fall wird "Value" auf 0 gesetzt, um anzuzeigen, dass das Licht ausgeschaltet ist. Außerdem wird "Value1" um 1 verringert, um die Anzahl der Ausschaltvorgänge zu zählen. Schließlich wird die GPIO-Ausgabe des LED-Pins auf "LOW" gesetzt, um das Licht auszuschalten. Damit die Endlosschleife nicht die CPU vollständig beansprucht, wird ein kurzer Schlaf von 0,3 Sekunden mit dem Befehl "time.sleep(0.3)" eingelegt, bevor die Schleife erneut durchlaufen wird. Schließlich wird die Funktion "light_thread" in einem separaten Thread ausgeführt, der mit dem Namen "light_t" definiert wird. Dies wird durch den Befehl "threading.Thread(target=light_thread)" erreicht. Dadurch wird die Funktion in einem separaten Thread ausgeführt, ohne den Hauptprozess des Programms zu blockieren.

Insgesamt handelt es sich hierbei um einen Code, der mithilfe der "threading"-Bibliothek einen separaten Thread erstellt, der den Status eines Lichtsensors abruft und die Beleuchtung entsprechend ein- oder ausschaltet. Durch die Verwendung von globalen Variablen innerhalb der Funktion wird sichergestellt, dass Änderungen an diesen Variablen auch außerhalb der Funktion sichtbar sind. Der kurze Schlaf zwischen den Schleifendurchläufen verhindert, dass die CPU übermäßig beansprucht wird.

3.2.3 Probleme

Eins der aufgetretene Probleme war, dass im ersten Versuch vom Code das Licht nur im Haus an ging, wenn man gedrückt hielt. Das Problem im vorherigen Code war, dass das Licht nur aufleuchtete, wenn man den Taster gedrückt hielt, und es ausging, wenn man ihn losließ. Dies lag daran, dass die Abfragefunktion "abfrage()" die ganze Zeit in einer Schleife ausgeführt wurde und nicht auf ein Signal vom Taster wartete. Im neuen Code wurde ein Thread namens "light_thread()" erstellt, der eine Endlosschleife ausführt und auf das Signal vom Taster wartet. Sobald der Taster gedrückt wird und Value gleich 0 ist, wird das Licht eingeschaltet, der Wert von Value auf 1 gesetzt und Value1 um eins erhöht. Wenn der Taster nochmals

gedrückt wird und Value gleich 1 ist, wird das Licht ausgeschaltet, der Wert von Value auf 0 gesetzt und Value1 um eins verringert. Dieses Verhalten ermöglicht es dem Nutzer, das Licht mit nur einem Tastendruck an- und auszuschalten.

Das Problem wurde also gelöst, indem ein Thread implementiert wurde, der auf das Signal vom Taster wartet und das Verhalten des Lichts basierend auf diesem Signal ändert. übermäßig beansprucht wird.

3.3Umsetzungsphase des Automatischen Außen Strahlers

3.3.1 Theoretische Umsetzung

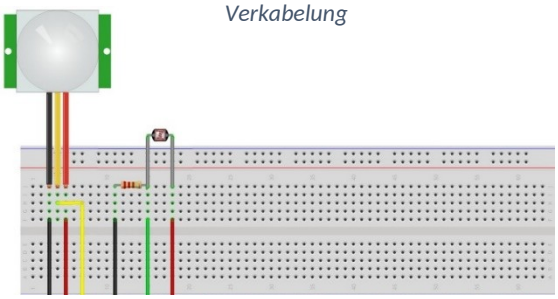


Abbildung 7: Verbildlichung der Verkabelung

Die Materialien belaufen sich bei dieser Funktion des Smarthomes um eine kleine Anzahl.
Wir benötigen: 1x Breadboard, 1x Diodenled und 4x Jumper Kabel. Um den Sensor anzuschließen, wird er zunächst einmal an einem Ground und 5V Pin des Extensionsboard angeschlossen. Dann wird der Input-Pin des Sensors an den GPIO-Pin 2 angeschlossen. Nun wird noch der Output-Pin in unserem Falle der GPIO-Pin 3 an eine 1k Ω am Breadboard angeschlossen, welche mit ihrer längeren Seite dann an der LED angeschlossen und ein mindestens 3.3V in reihe der kürzeren Seite angesteckt(siehe ABB. 7). Zuletzt wird nur noch ein Kabel mit Ground an die kürzere Seite der Diodenled anschlossen. Diese Funktion steuert nun bei einer Bewegung im Radius von 3 Meter des Sensors eine LED, die für 30 Sekunden ohne weitere Erkennung an bleibt. Wird sich jedoch dauerhaft vor dem Sensors bewegt wird die LED so lange an bleiben, bis diese Bewegung entweder außerhalb des Radius ist oder es keine Bewegung stattfindet.

3.3.2 Code

```
try:
    while True:
        if GPIO.input(motion_sensor_pin):
            print("Bewegung erkannt!")
            GPIO.output(led_pin, GPIO.HIGH)
            time.sleep(1)
            GPIO.output(led_pin, GPIO.LOW)
        time.sleep(0.1)
except KeyboardInterrupt:
```

Abbildung 8: Die Schleife mit der Hauptfunktion

Als erstes werden die Pins für den Bewegungssensor und die LED definiert. Der Bewegungssensor wird an Pin 2 angeschlossen und die LED an Pin 3. Beide Pins werden dann konfiguriert, indem sie entweder als Eingang der Ausgang definiert werden. Dann beginnt die Schleife für die Bewegungserkennung. Diese Schleife überwacht kontinuierlich den Bewegungssensor, indem sie die Eingabe des Bewegungssensors in jeder Iteration überprüft. Wenn der Sensor eine Bewegung erkennt, wird die Bedingung if

GPIO.input(motion_sensor_pin) erfüllt und der folgende Codeblock wird ausgeführt. In diesem Codeblock wird zuerst eine Meldung "Bewegung erkannt!" ausgegeben, um den Benutzer zu informieren. Dann wird der Pin von GPIO 3 als Ausgang gesetzt, um die LED zum Leuchten zu bringen. Die LED bleibt dann für eine Sekunde an, indem time.sleep(1) aufgerufen wird. Schließlich wird der Pin von GPIO 3 wieder als Eingang gesetzt, um die LED auszuschalten. Die Schleife geht dann weiter, um den Bewegungssensor weiterhin zu überwachen. Der Code enthält auch eine Verzögerung von 0,1 Sekunden zwischen den Schleifendurchläufen, um die CPU-Auslastung zu verringern und die Stromversorgung des Raspberry Pi zu schonen. Zusammenfassend ist dies ein Beispiel für einen Bewegungssensor angeschlossen an einem Raspberry Pi, um eine LED-Leuchte bei Bewegungserkennung für 15 Sekunde anzuschalten. Mit kleinen Anpassungen, wie z.B. eine Verlängerung der Verzögerung oder Anpassung des Zeitraums, in dem die LED an bleiben soll, kann dieser Code auch als Außenstrahler genutzt werden. Durch die Verwendung von time.sleep kann zudem die Länge der Leuchtdauer von der LED gesteuert werden. Hierbei sollte beachtet werden, dass der Bewegungssensor so positioniert wird, dass er Bewegungen von Personen erkennt, jedoch keine Fehlauslösungen durch Tiere, Pflanzenbewegungen oder Ähnliches entstehen.

3.3.3 Probleme

Es sind beim Testen des Programmes zwei Fehler entstanden. Der erste Fehler belief sich auf Fehlreaktionen des Sensors und somit zu unnötigem Energie Verlust. Einer der beiden gefunden Möglichkeiten von mir wäre darin, den Bewegungssensor so zu positionieren, dass er nur Bewegungen von Personen erkennt, und andere Faktoren, die den Sensor auslösen könnten, minimiert werden. Eine weitere Möglichkeit besteht darin, die Empfindlichkeit des Sensors anzupassen über den Regler, um Fehlauslösungen zu reduzieren. Der zweite Fehler kam zustande, indem es passieren könnte das der Raspberry Pi sich überhitzten könnte, da diesen Programm dauerhaft laufen würde, hierfür könnte man den schon verbauten Lichtsensor nutzten um ihn nur bei Dunkelheit aufleuchten lassen.

3.4 Umsetzungsphase der Automatisierte und manuell steuerbare Jalousien

3.4.1 Theoretische Umsetzung

Diese Aufgabe war die Komplexeste der Drei Funktion des Smarthome und benötigt deswegen auch die meisten Materialien: 24x Jumper Kabel, 1x Breadboard, 1x Lichtsensor und pro Fenster ein Stepmotor. Ich habe mich für dieses Projekt absichtlich für den Stepmotor

Modell 28BYJ-48 entschieden, da dieser sehr klein ist und präzise Stepps ermöglicht. Jedoch auch Möglichkeiten wären gewesen der Motor Nema17. Zuallererst wird der Motor erstmal an die dazu gelieferte Modulplatine angesteckt. Nun muss man ihn mit mindestens 5V und Ground am dem Extensionsboard anstecken. Der nächste Schritt wäre die 4 Input Pins noch anzustecken, hierfür habe ich die GPIO-Pins 06, 13, 19 und 26 benutzt, da diese alle nebeneinander liegen. Jetzt wird der Lichtsensor angesteckt, dazu muss man ihn wieder an 5V und Ground anpinnen und ein Input-Pin belegen in meinem Falle der Pin 05. Es wurde sich Speziell wegen der Genauigkeit, Mobilität und des Preises des Lichtsensor das Modelle LM393 entschieden, alternativ und einem hören Budget wäre auch das Modell RBS13321 möglich . Da das Projekt nur um ein Miniatur Haus geht habe ich den Regler der Empfindlichkeit des Sensors auf das Minimum reduziert damit er nicht so schnell anschlägt und fehlerhaft auslöst. Nun Schließen wir drei Buttons an ein Breadboard die die Funktion haben, dass System zu Stoppen und es Manuell steuern zu lassen. Dafür müssen alle Knöpfe mit einem Ground, 5V und einem GPIO-verbunden werden. Dafür benutzten wir die drei nebeneinander liegenden Pins 17, 27 und 22. Beim Programm Start wird nun die Automatische Schaltung erstmal aktiviert, diese funktioniert indem Licht Einstrahlung auf den Sensor trifft und somit den Motor nach Links drehen und erst wenn diese Licht Strahlung weg ist, wird er nach rechts gedreht. Ob Licht vorhanden ist oder eben nicht wird vom Programm jede 15 Sekunden einmal geprüft, falls der Wert vom Lichtsensor gleichbleibt, wird so lange auch nichts passieren. Um in den Manuellen Modus zu wechseln, muss man während des 15 Sekunden Countdowns, der angezeigt wird im Programm den Mittleren also den Knopf drücken, der auf Pin 27 liegt. Nun ist das Programm im Manuellen Modus und kann je nach Belieben von den beiden Knöpfen entweder nach Links oder Rechts gedreht werden.

3.4.2 Code

```
import time
import RPi.GPIO as GPIO
import threading

GPIO.setwarnings(False)

knopf = 27
right_pin = 22
left_pin = 17

sensor_pin = 5
motor_pins = [6, 13, 19, 26]
stepper_sequence = [[1, 0, 0, 1],
                    [1, 0, 0, 0],
                    [1, 1, 0, 0],
                    [0, 1, 0, 0],
                    [0, 1, 1, 0],
                    [0, 0, 1, 0],
                    [0, 0, 1, 1],
                    [0, 0, 0, 1]]

global Value
global Value1
global trash
trash = True
Value = 0
Value1 = 0
```

Abbildung 8: Pin-Festlegung

Das Programm beginnt damit, einige GPIO-Pins zu importieren und zu initialisieren
tzt und der LED-Pin ist auf 6 gesetzt. Ein weiterer
erungsbutton dient. Es gibt auch zwei weitere
nanuellen Steuerungsbutton steuern. Ein globales
Als nächstes wird ein MotorThread-Objekt
ttmotors enthält. Der MotorThread-Thread wird
die Pins 6, 13, 19 und 26 gesteuert. Diese Pins

werden im Codeabschnitt "for pin in motor_pins" initialisiert und auf 0 gesetzt. Das bedeutet, dass der Motor zu Beginn stillsteht.

Der MotorThread enthält eine run()-Methode(schaue ABB. 9), die die Hauptlogik des Programms enthält. Zunächst wird die aktuelle Sensorstatus abgefragt, um zu bestimmen, ob Licht auf den Lichtsensor fällt oder nicht. Wenn der aktuelle Sensorstatus vom vorherigen Status abweicht, wird geprüft, ob das Lichtstrahlung vorhanden ist oder nicht. Wenn der Sensor Licht erkennt, wird der Motor in die entgegengesetzte Richtung gedreht, wenn es dunkel ist, wird der Motor in die andere Richtung gedreht. Während der Motor in Bewegung ist, wird die Methode time.monotonic() verwendet, um den Startzeitpunkt zu speichern. Die Schleife wird so lange ausgeführt, bis das Zeitintervall von 15 Sekunden abgelaufen ist. In jedem Schleifendurchlauf wird der Schrittmotor um eine Stufe weitergedreht. Das passiert durch die Verwendung einer Schrittabfolge, die im Codeabschnitt "stepper_sequence" definiert ist. Es handelt sich hierbei um eine Liste mit acht Elementen, die die Pins des Schrittmotors in der Reihenfolge enthält, in der sie aktiviert werden müssen.

```
def run(self):
    step = 0
    direction = 1
    motor_running_time = 15
    last_sensor_state = None

    while True:
        current_sensor_state = GPIO.input(sensor_pin)

        if current_sensor_state != last_sensor_state:
            last_sensor_state = current_sensor_state

            if current_sensor_state == GPIO.HIGH:
                print("Licht erkannt - Schrittmotor dreht nach links")
                direction = -1
            else:
                print("Kein Licht - Schrittmotor dreht nach rechts")
                direction = 1

            motor_start_time = time.monotonic()
            while (time.monotonic() - motor_start_time) < motor_running_time:
                with self.pause_cond:
                    while self.paused:
                        self.pause_cond.wait()
                for i in range(8):
                    for pin in range(4):
                        GPIO.output(motor_pins[pin], stepper_sequence[step][pin])
                    step += direction
                    if step >= 8:
                        step = 0
                    elif step < 0:
                        step = 7
                    time.sleep(0.001)
```

Abbildung 9: Def Run

Nachdem der Motor 15 Sek. langgedreht hat, wird eine Pause von 15 Sek. eingelegt, um den Schrittmotor zu stoppen und zu warten, bis sich der Sensorstatus wieder ändert.

Es gibt auch eine Methode pause(), mit der der MotorThread-Thread pausiert werden kann, und eine Methode resume(), mit der der MotorThread-Thread fortgesetzt werden kann. Diese Methoden werden verwendet, um den manuellen Modus des Programms zu implementieren.

```
def pause(self):
    self.paused = True
    print("MotorThread pausiert")

def resume(self):
    self.paused = False
    with self.pause_cond:
        self.pause_cond.notify()
    print("MotorThread fortgesetzt")
```

Abbildung 10: Code um die Module zu wechseln

Abbildung 11: Die Callback Funktion

```
def button_callback(channel):
    global trash
    if trash:
        trash = False
        print("Überprüfung ausgeschaltet")
        motor_t.pause()
    else:
        trash = True
        print("Überprüfung angeschaltet")
        motor_t.resume()
```

Die Methode button_callback(channel) (siehe ABB.11) wird aufgerufen, wenn der manuelle Steuerungsbutton gedrückt wird. Wenn der manuelle Modus aktiviert ist, wird der MotorThread-Thread pausiert und die

Überprüfung wird gestoppt. Wenn der manuelle Modus deaktiviert ist, wird der MotorThread-Thread fortgesetzt. Der MotorThread liest die Sollgeschwindigkeit aus dem ControlThread und passt die tatsächliche Geschwindigkeit des Motors entsprechend an. Der Motor wird so lange in Betrieb bleiben, bis die Jalousien zum Stillstand kommt oder der manuelle Modus wieder aktiviert wird.

Wenn die Jalousien sich im autonomen Modus befinden, wird der ControlThread den MotorThread anweisen, den Motor basierend auf den Daten der Sensoren zu steuern. Zusätzlich zum MotorThread gibt es noch einen SensorThread, der die Daten von verschiedenen Sensoren des Jalousinen liest und an den ControlThread weitergibt. Der ControlThread wird dann Entscheidungen auf der Grundlage dieser Daten treffen und den MotorThread entsprechend anweisen.

Insgesamt arbeiten diese Threads zusammen, um sicherzustellen, dass die Jalousien auf eine effiziente und zuverlässige Weise funktioniert, unabhängig davon, ob er im manuellen oder autonomen Modus arbeitet.

3.4.3 Probleme

Da dieses Programm ein sehr Komplexes war sind natürlich auch hier einige Fehler entstanden. Eins der größten Probleme, was das Programm anfangs aufwies, war die Manuelle Steuerung einzubauen, da diese sich mit dem Automatisierten Prozess behinderte und somit dauerhaft zu Fehlmeldungen und gleichzeitigen Handlung führte. Um dieses Problem zu lösen, baute ich den Knopf ein, der die Automatisierung ausschaltete. Dies jedoch führte dazu das nach dem wieder anschalten des Automatisierung Modus dauerhaft lief so baute ich ein, dass der Knopf zur manuellen Schaltung nur funktionieren kann, während der 15 Sek Countdown läuft. Ein auch schwerwiegender Fehler war das, falls es zweimal zum gleichen Zustand des Lichtsensor kam, drehte sich der Motor und somit die Jalousien Trotzdem. Um diesen Fehler zu beheben, wurde eine Value hinzugefügt die überprüft, ob es den gleichen Wert wie vorher hat. Somit wird es nur zur eine Motor Drehung kommen, wenn ein anderer Wert als vorher erkannt wird. Schließlich gab es noch ein Problem, um zu wissen ob grade der Manuelle oder Automatisierte Modus eingeschaltet ist. Dafür wurde im Code einfach eine Wiedergabe hinzugefügt in welchem Modus er sich grade befindet.

4. Fazit

Die Planung und Umsetzung eines Smarthome-Systems mit einem Raspberry Pi war ein äußerst spannendes und lehrreiches Projekt, das uns zahlreiche Herausforderungen und Möglichkeiten bot.

Wir waren in der Lage, mithilfe von Bewegungssensoren, Lichtsensoren, einem Stepmotor und einem Touchsensor ein echtes Haus sehr gut nachzubilden und uns so ein genaues Bild davon zu machen, wie das System in unserem eigenen Haus funktionieren und aussehen würde. Dies war nicht nur faszinierend, sondern auch äußerst lehrreich, da ich dabei unsere Fähigkeiten in den Bereichen Elektrotechnik und Programmierung erweitern konnten und viele wertvolle Erfahrungen sammelten. Besonders wichtig war jedoch die gründliche Planung des Projekts. Durch eine umfassende Vorab-Analyse und die Identifizierung aller notwendigen Komponenten konnten wir ein solides Fundament für den Bau des Smarthome-Systems schaffen. Dies erleichterte uns die Umsetzung des Projekts erheblich und half uns, zahlreiche Herausforderungen zu meistern. Vor allem die Zeichnung erwies sich als sehr nützlich da wir dadurch die genaue Masse für die Aktoren und Sensoren mit ein berechnen konnten. Während der Umsetzung traten jedoch auch Schwierigkeiten auf, wie beispielsweise die Installation und Konfiguration von Bauteilen oder die Programmierung der verschiedenen Komponenten. Diese Herausforderungen konnten jedoch erfolgreich gemeistert werden, da wir als Familie zusammengearbeitet und die individuellen Stärken und Fähigkeiten jedes einzelnen eingebracht haben. Durch diese Zusammenarbeit konnten wir das Smarthome-System gemeinsam erfolgreich fertigstellen und waren stolz auf das Ergebnis unserer Arbeit. Nach Abschluss des Projekts können ich und meine Familie jedem, der sich für das Thema interessiert und Spaß an der Umsetzung hat, nur empfehlen, ein ähnliches Projekt in Angriff zu nehmen. Durch unsere Erfahrungen haben wir gelernt, welche Arbeit hinter einem Smarthome-System steckt und welche Dinge man beachten muss. Dieses Wissen können wir nun auch in unserem eigenen Haus anwenden und dadurch von den Vorteilen eines solchen Systems profitieren.

Insgesamt war die Fertigstellung des Smarthome-Systems ein bedeutender Meilenstein in unserem Projekt. Das System bot zahlreiche Vorteile, wie beispielsweise eine verbesserte Energieeffizienz, erhöhte Sicherheit und eine verbesserte Benutzerfreundlichkeit. Wir waren begeistert von der Funktionalität und den Möglichkeiten, die uns das System bot. Darüber hinaus war es ein großartiges Beispiel dafür, wie eine Zusammenarbeit innerhalb der Familie zu einem erfolgreichen Projekt führen kann. Abschließend können wir sagen, dass die Planung und Umsetzung eines Smarthome-Systems mit einem Raspberry Pi ein äußerst lohnendes und erfüllendes Projekt ist, das uns nicht nur viel Spaß gemacht hat, sondern uns auch viele wertvolle Erfahrungen und Erkenntnisse vermittelt hat. Wir sind stolz auf das, was wir erreicht haben, und freuen uns darauf, unsere neuen Fähigkeiten und Kenntnisse in zukünftigen Projekten anzuwenden.

5. Literaturverzeichnis

Dies sind die Quellen, woher die Informationen dieser Dokumentation stammen.

Erklärung verschiedenster Sensoren:

https://electreeks.de/project/raspberry-pir-hc-sr-501-installation-und-funktion/

https://codingworld.io/touch0-einfuehrung-touch-sensor/

https://tutorials-raspberrypi.de/raspberry-pi-helligkeitssensor-fotowiderstand/

https://www.laub-home.de/wiki/Raspberry_Pi_BH1750_Helligkeitssensor

https://www.franzis.de/media/pdf/f6/ec/2e/60490-1_LP_Sensoren_am_Raspberry_Pi.pdf

https://www.youtube.com/watch?v=-wm05ODt2XA

Erklärung für die Knöpfe

https://www.elektronik-kompendium.de/sites/raspberry-pi/2110081.htm

Erklärung für die LED

https://cool-web.de/raspberry/interne-raspberry-pullup-vorwiderstaende-nutzen.htm

https://funduinoshop.com/blinkende-led-am-raspberry-pi-mikrocontroller

Erklärung des Raspberry Pi und seine Funktionen

https://cdn-reichelt.de/documents/datenblatt/A300/BEGINNERSGUIDE-4THED-GER_V2.pdf

https://www.futurezone.de/deals/article232474019/was-ist-ein-raspberry-pi-einfach-und-verstaendlich-erklaert.html

https://jankarres.de/2015/02/raspberry-pi-gpio-schnittstelle-erklaert/

https://www.giga.de/artikel/was-ist-raspberry-pi-einfach-erklaert--cwr5c2fqr7

https://www.youtube.com/watch?v=2Kl0yKRJ-FM

https://www.datenreise.de/raspberry-pi-wie-verwendet-man-ein-breadboard-steckplatine-anleitung/

Erklärung Stepmotor

https://www.electronicwings.com/raspberry-pi/stepper-motor-interfacing-with-raspberry-pi

https://www.kollmorgen.com/de-de/blogs/wie-funktioniert-ein-schrittmotor